

Самостоятельный учебный проект

Распознавание именованных сущностей в научных текстах

Сравнительный анализ архитектур на датасете SciERC

Иван Четвергов
chitiverg@gmail.com

Май 2026

Код проекта: <https://github.com/ivanchetvergov/SciNER>

Содержание

Введение	3
1 Данные	4
1.1 Датасет SciERC	4
1.2 Типы сущностей	4
1.3 BIO-разметка	4
1.4 Статистика	4
2 Архитектуры моделей	5
2.1 Энкодеры	5
2.2 Классификационные головы	5
2.3 Взвешивание классов	5
2.4 Полный список моделей	6
3 Препроцессинг данных	7
3.1 Эксперимент 1 — базовый препроцессинг	7
3.2 Эксперимент 2 — улучшенный препроцессинг	7
3.2.1 Аугментация заменой сущностей	7
3.2.2 Межпредложенческий контекст	7
3.3 Метрика оценки	8
4 Условия обучения	9
4.1 Гиперпараметры	9
4.2 Инфраструктура	9
5 Результаты	10
5.1 Эксперимент 1 — базовый препроцессинг	10
5.1.1 Влияние взвешивания классов	10
5.1.2 Влияние научного предобучения	10
5.1.3 Влияние архитектуры головы	11
5.2 Кривые обучения	11
5.3 Анализ ошибок	11
5.4 Эксперимент 2 — улучшенный препроцессинг	12
Заключение	13

Введение

Распознавание именованных сущностей (Named Entity Recognition, NER) — базовая задача обработки естественного языка, которая состоит в выделении в тексте упоминаний объектов заданных категорий и их классификации. В научных текстах набор категорий существенно отличается от новостных или общелингвистических корпусов: вместо персон и организаций исследователям важны методы, метрики, датасеты и научные концепции.

Целью данного проекта является сравнительное исследование нейросетевых архитектур на задаче NER научных текстов. В качестве датасета использован SciERC [1] — корпус из 500 аннотированных научных абстрактов по компьютерным наукам. Набор содержит 6 типов именованных сущностей, что делает его нетривиальным: классы семантически близки и имеют существенный дисбаланс по частоте.

В работе решаются следующие задачи:

1. Реализация и обучение 10 моделей на базе предобученных трансформеров (BERT, RoBERTa, SciBERT) с различными классификационными головами.
2. Исследование влияния взвешивания классов, архитектуры головы и качества предобучения энкодера на результат.
3. Оценка влияния улучшенного препроцессинга данных: аугментации через замену сущностей и расширения контекста через соседние предложения.

1 Данные

1.1 Датасет SciERC

SciERC [1] — корпус научных абстрактов из 12 конференций по ИИ (ACL, EMNLP, NeurIPS и др.). Датасет содержит 500 документов, размеченных на уровне именованных сущностей, отношений и кореференций. В данной работе используется только задача NER.

Стандартное разбиение: 350 документов в обучающей выборке, 50 в валидационной, 100 в тестовой.

1.2 Типы сущностей

Датасет определяет 6 типов именованных сущностей (таблица 1).

Таблица 1: Типы сущностей в SciERC

Тип	Описание	Пример
Task	Задача или проблема	<i>machine translation</i>
Method	Метод, модель, система	<i>BERT, attention mechanism</i>
Metric	Метрика оценки	<i>F1 score, BLEU</i>
Material	Датасет, ресурс, материал	<i>Penn Treebank, word embeddings</i>
OtherScientificTerm	Прочий научный термин	<i>feature, embedding</i>
Generic	Общий термин, сложно категоризировать	<i>approach, model</i>

1.3 BIO-разметка

Сущности кодируются в формате BIO (Beginning–Inside–Outside):

- B-X — первый token сущности типа X,
- I-X — продолжение сущности типа X,
- O — token вне именованной сущности.

Итого 13 меток: O плюс B-/I- для каждого из 6 типов. Классы сущностей несбалансированы: метка O составляет большинство токенов, а такие типы, как Metric и Task, встречаются значительно реже остальных.

1.4 Статистика

Дисбаланс классов — ключевая особенность датасета, влияющая на выбор функции потерь. В разделе 4 описано, как это учитывается при обучении.

2 Архитектуры моделей

Все модели строятся по схеме «предобученный трансформерный энкодер + классификационная голова». Энкодер принимает последовательность токенов и возвращает контекстуализированные представления; голова предсказывает метку для каждого токена.

2.1 Энкодеры

BERT [2] (bert-base-cased) — предобученная модель на общем корпусе (BooksCorpus + Wikipedia), 12 слоёв, 768 скрытых размерностей.

RoBERTa [3] (roberta-base) — улучшенная версия BERT с более агрессивным обучением (больше данных, динамическое маскирование, без NSP), аналогичная архитектура.

SciBERT [4] (allenai/scibert_scivocab_cased) — BERT, дообученный на 1.14 млн научных статей из Semantic Scholar. Имеет собственный научный словарь. Ожидается, превосходит общедоменные модели на научных текстах.

2.2 Классификационные головы

Linear. Дропаут $\rightarrow$ линейный слой $\mathbb{R}^H \rightarrow \mathbb{R}^K$, где $H = 768$ — размер скрытого состояния, $K = 13$ — число меток. Обучается с кросс-энтропией.

MLP. Дропаут $\rightarrow$ Linear($H, 256$) $\rightarrow$ GELU $\rightarrow$ Linear($256, K$). Добавляет нелинейность перед финальной классификацией.

Linear + CRF. Линейный слой формирует эмиссионные потенциалы; CRF-слой [5] моделирует зависимости между соседними метками и применяет алгоритм Витерби при декодировании. Функция потерь — отрицательное логарифмическое правдоподобие последовательности.

Concat-4. Конкатенация скрытых состояний последних 4 слоёв трансформера $\mathbb{R}^{4H}$, затем дропаут и линейный слой. Позволяет модели использовать признаки разного уровня абстракции.

2.3 Взвешивание классов

Для компенсации дисбаланса применяются веса классов:

$$w_c = \sqrt{\frac{N}{K \cdot n_c}},$$

где N — общее число токенов в обучающей выборке, n_c — число токенов класса c . Квадратный корень смягчает взвешивание по сравнению с обратной частотой, предотвращая чрезмерное наказание частых классов. Каждая модель обучается в двух вариантах: с весами и без.

2.4 Полный список моделей

Таблица 2: Модели, участвующие в эксперименте

Название	Энкодер	Голова	Веса
bert_linear_noweight	BERT	Linear	нет
bert_linear	BERT	Linear	да
roberta_linear_noweight	RoBERTa	Linear	нет
roberta_linear	RoBERTa	Linear	да
scibert_linear_noweight	SciBERT	Linear	нет
scibert_linear	SciBERT	Linear	да
scibert_mlp	SciBERT	MLP	нет
scibert_linear_crf_noweight	SciBERT	Linear+CRF	нет
scibert_linear_crf	SciBERT	Linear+CRF	да
scibert_concat4	SciBERT	Concat-4	нет

3 Препроцессинг данных

Проводится два эксперимента с разными условиями подготовки данных.

3.1 Эксперимент 1 — базовый препроцессинг

Токенизация. Каждое предложение токенизируется независимо с помощью `HuggingFace AutoTokenizer (is_split_into_words=True)`. Поскольку `WordPiece` и `BPE` разбивают слова на субтокены, метки выравниваются по правилу: первый субтокен слова получает метку слова, последующие субтокены — метку `I-X` (если слово начинается с `B-X`) или ту же метку `I-X`. Специальные токены `[CLS]`, `[SEP]` получают маску `-100` и не участвуют в вычислении функции потерь.

Динамический паддинг. Паддинг выполняется не при токенизации, а при сборке батча: последовательности дополняются до длины самой длинной в батче. Это уменьшает вычислительную нагрузку по сравнению с паддингом до фиксированной длины 512.

Аугментация и межпредложенческий контекст не применяются.

3.2 Эксперимент 2 — улучшенный препроцессинг

Все изменения касаются только обучающей выборки; валидационная и тестовая обрабатываются так же, как в эксперименте 1 (кроме контекстного окна).

3.2.1 Аугментация заменой сущностей

Строится словарь сущностей по типу: для каждого типа собираются все `span`'ы из обучающей выборки. Для каждого предложения каждый `span` заменяется с вероятностью $p = 0.5$ случайным `span`'ом того же типа из словаря. Предложения, в которых произошла хотя бы одна замена, добавляются в обучающую выборку как новые примеры (оригиналы сохраняются). Метод позволяет модели видеть больше контекстов для каждого типа сущности и снижает переобучение на конкретные упоминания.

3.2.2 Межпредложенческий контекст

Для каждого предложения к нему приписываются соседние предложения из того же документа (окно $w = 1$): одно предшествующее и одно следующее. Токены контекстных предложений получают метку-заглушку,

которая отображается в маску -100, поэтому функция потерь вычисляется только по центральному предложению. BERT при этом кодирует токены с учётом соседнего контекста, что даёт модели более богатые представления на границах предложений.

Схема для предложения i в документе $[s_1, \dots, s_n]$:

$$\underbrace{s_{i-1}}_{\text{контекст}} \quad \underbrace{s_i}_{\text{цель}} \quad \underbrace{s_{i+1}}_{\text{контекст}}$$

3.3 Метрика оценки

Качество оценивается по entity-level macro F1 (библиотека seqeval). Сущность считается правильно распознанной только при точном совпадении границ span'а и типа. Macro F1 усредняет F1 по всем 6 типам с равными весами, что делает метрику чувствительной к редким классам.

4 Условия обучения

4.1 Гиперпараметры

Все модели обучаются с едиными гиперпараметрами (таблица 3).

Таблица 3: Общие гиперпараметры обучения

Параметр	Значение
Оптимизатор	AdamW, weight decay = 0.01
Learning rate	2×10^{-5} (энкодер и голова)
Learning rate (CRF)	10^{-3} (только CRF-слой)
Планировщик	Linear warmup (1 эпоха) + линейный decay
Batch size	16 (8 для scibert_concat4)
Максимальное число эпох	10
Early stopping	patience = 3 (по dev macro F1)
Случайное зерно	42
Смешанная точность	AMP (fp16) на GPU

4.2 Инфраструктура

Обучение проводилось на платформе Kaggle с двумя GPU NVIDIA T4 (по 16 GB памяти). Для использования двух GPU применялся `torch.nn.DataParallel` (за исключением scibert_concat4, отключённый из-за несовместимости с механизмом захвата скрытых состояний через forward-хуки).

Лучший чекпоинт по dev F1 сохраняется через early stopping; тестовая оценка производится на лучшей эпохе.

5 Результаты

5.1 Эксперимент 1 — базовый препроцессинг

Результаты всех моделей на тестовой выборке приведены в таблице 4.

Таблица 4: Macro F1 и per-entity F1 на тесте (эксперимент 1)

Модель	Macro	Gen.	Mat.	Met.	Metr.	OST	Task
bert_linear_noweight	0.604	0.714	0.557	0.681	0.573	0.531	0.565
bert_linear	0.594	0.656	0.568	0.676	0.593	0.523	0.546
roberta_linear_noweight	0.591	0.692	0.561	0.671	0.548	0.519	0.556
roberta_linear	0.585	0.673	0.557	0.651	0.593	0.531	0.505
scibert_linear_noweight	0.646	0.712	0.659	0.749	0.618	0.566	0.571
scibert_linear	0.603	0.688	0.615	0.725	0.533	0.545	0.515
scibert_mlp	0.644	0.708	0.665	0.746	0.615	0.577	0.553
scibert_linear_crf_noweight	0.628	0.721	0.628	0.723	0.543	0.570	0.581
scibert_linear_crf	0.628	0.721	0.628	0.723	0.543	0.570	0.581
scibert_concat4	0.644	0.737	0.661	0.751	0.573	0.579	0.561

Gen. = Generic, Mat. = Material, Met. = Method, Metr. = Metric, OST = OtherScientificTerm

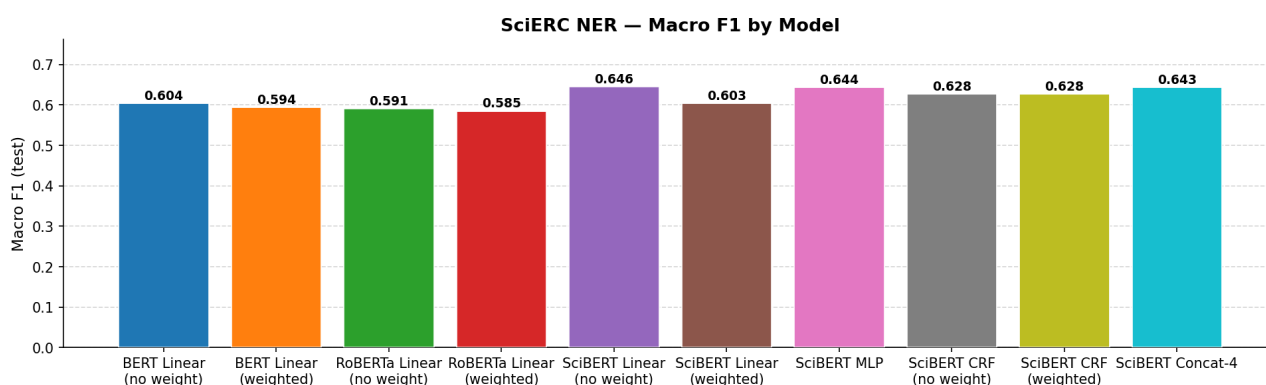


Рис. 1: Macro F1 по моделям (эксперимент 1)

5.1.1 Влияние взвешивания классов

Во всех случаях вариант без весов превосходит взвешенный: `scibert_linear_noweight` (0.646) против `scibert_linear` (0.603), аналогично для BERT и RoBERTa. Вероятная причина: взвешивание избыточно штрафует класс 0 и вынуждает модель «видеть» сущности там, где их нет, снижая точность.

5.1.2 Влияние научного предобучения

SciBERT-модели стабильно превосходят BERT и RoBERTa. Лучший BERT (`bert_linear_noweight`, 0.604) уступает лучшему SciBERT (`scibert_linear_noweight`, 0.646) на 4.2 п.п. — ожидаемый результат: специализированное предобучение важно для научного домена.

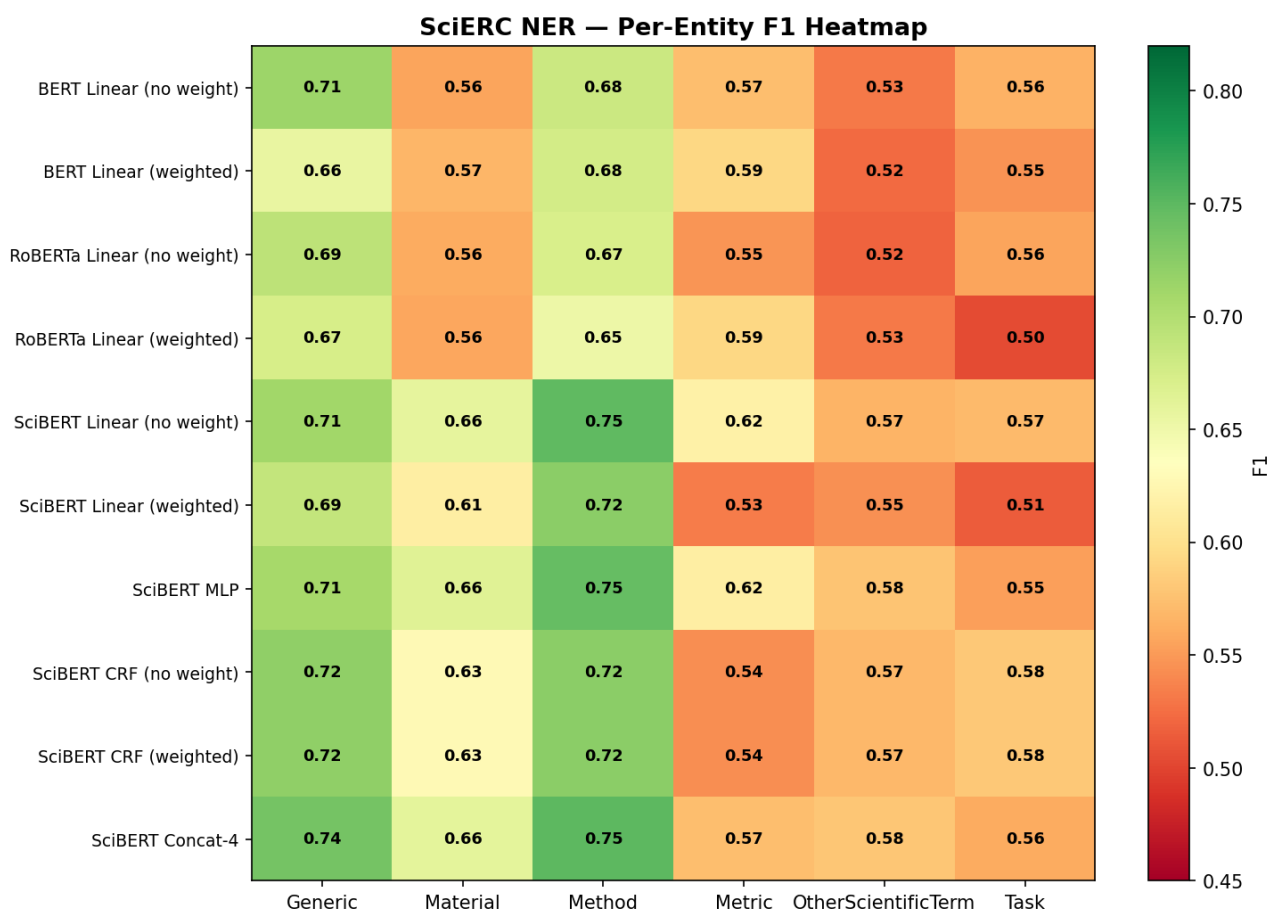


Рис. 2: Тепловая карта per-entity F1 (эксперимент 1)

5.1.3 Влияние архитектуры головы

MLP и Concat-4 незначительно уступают простой линейной голове без весов (0.644 vs 0.646). CRF не даёт прироста: 0.628, что ниже лучшего SciBERT-baseline. Возможная причина — CRF усложняет оптимизацию без существенной структурной выгоды при коротких и несложных span'ах научных сущностей.

5.2 Кривые обучения

SciBERT-модели сходятся быстрее и стабильнее BERT/RoBERTa. RoBERTa демонстрирует заметную нестабильность на первых эпохах.

5.3 Анализ ошибок

Матрицы ошибок показывают (рис. 4): основная проблема — пропуски (столбец «Missed»), а не путаница между типами. Чаще всего пропускаются Metric и Task — редкие классы. Путаница между типами сосредоточена на парах Generic↔Method и Generic↔OtherScientificTerm, что семантически объяснимо.

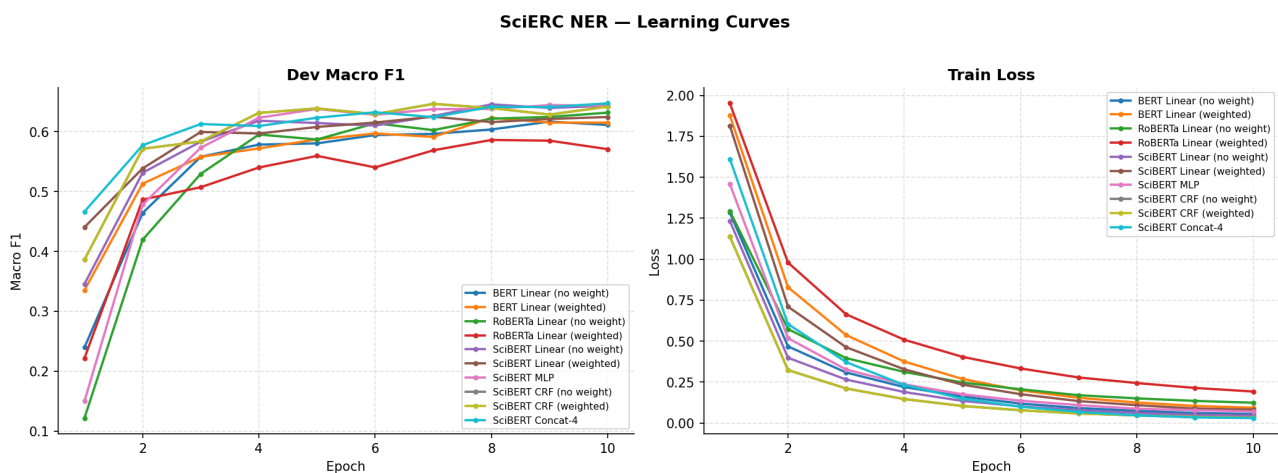


Рис. 3: Кривые обучения по эпохам (эксперимент 1)

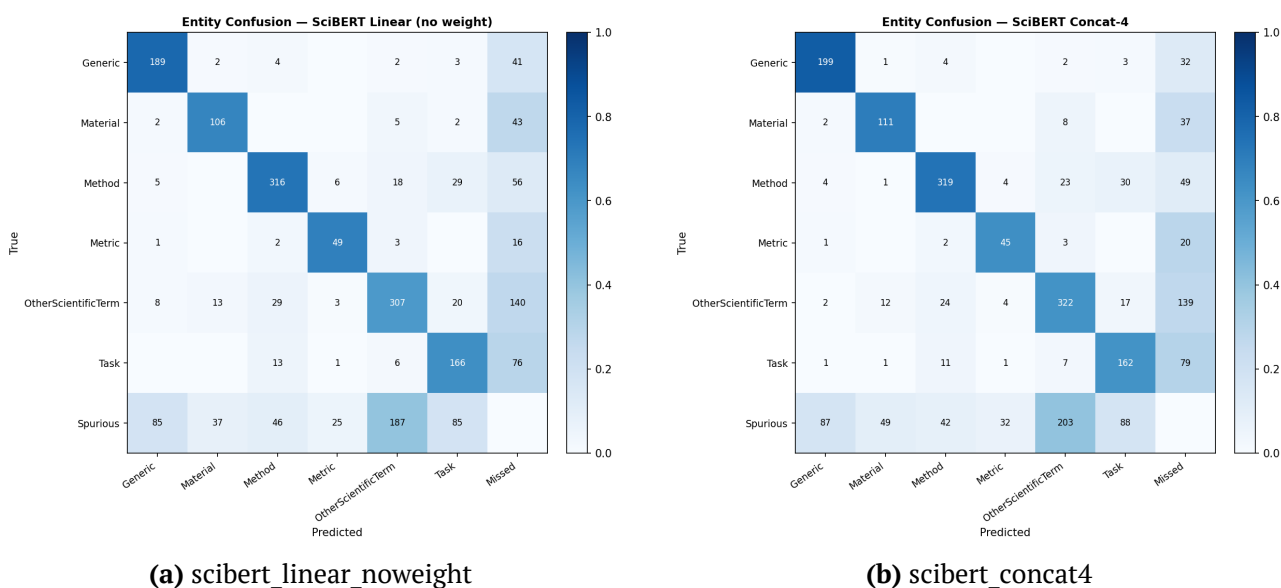


Рис. 4: Матрицы ошибок лучших моделей (нормировано по строкам)

5.4 Эксперимент 2 — улучшенный препроцессинг

Результаты эксперимента 2 (с аугментацией и межпредложенческим контекстом) будут добавлены после завершения обучения.

Заключение

В работе проведено сравнительное исследование 10 нейросетевых архитектур на задаче NER в научных текстах (датасет SciERC).

Основные результаты. Лучший результат в базовом эксперименте показала модель `scibert_linear_noweight` — $\text{macro F1} = 0.646$. SciBERT стабильно превосходит BERT и RoBERTa за счёт научного предобучения. Простая линейная голова без взвешивания классов оказалась конкурентоспособнее сложных вариантов (MLP, CRF, Concat-4).

Сильные стороны.

- Контролируемое сравнение: все модели обучены с одинаковыми гиперпараметрами, единственная переменная — архитектура.
- Полный пайплайн: от данных до confusion matrix и кривых обучения.
- Воспроизводимость: фиксированный seed, реестр прогонов.

Слабые стороны.

- Одна случайная инициализация на модель — нет доверительных интервалов.
- CRF-модели дали идентичные результаты в двух вариантах (вероятно, технический артефакт Kaggle-сессии).
- Редкие типы (Metric, Task) остаются сложными для всех моделей.

Направления развития.

- Span-based модели (DyGIE++, SpanBERT) вместо token-level BIO — качественно другой подход к границам сущностей.
- Joint NER+RE обучение, поскольку SciERC содержит разметку отношений.
- Более крупные энкодеры (SciBERT-large, SPECTER).

Список литературы

- [1] Luan, Y., He, L., Ostendorf, M., & Hajishirzi, H. (2018). *Multi-Task Identification of Entities, Relations, and Coreference for Scientific Knowledge Graph Construction*. EMNLP 2018.
- [2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL 2019.
- [3] Liu, Y., Ott, M., Goyal, N., et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv:1907.11692.
- [4] Beltagy, I., Lo, K., & Cohan, A. (2019). *SciBERT: A Pretrained Language Model for Scientific Text*. EMNLP 2019.
- [5] Lafferty, J., McCallum, A., & Pereira, F. (2001). *Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data*. ICML 2001.